# Yüküm

UYGULAMA PLANI – Hangi sırayla yapıyoruz, nasıl birleştiriyoruz?

Küçükten büyüğe sıralı yapım sırası • Backend + Veritabanı + iOS bağlama kuralları • Notlarla eşleşme kontrolü

---

Bu doküman **Yukum_Teknik_Rapor.pdf** dosyasının devamıdır. Teknik Rapor “ne ve neden”i, bu plan **“hangi sırayla ve nasıl birleştirilerek”** sorusunu anlatır.
**FİNAL (Sürüm 3) – bağlayıcı sıralama bu dokümandadır** • Tarih: 20 Ekim 2026 • Çalışma şekli: Backend + veritabanı birlikte, iOS’u kullanıcı yazar.

BAŞLANGIÇ

## Planın Haritası: Ne Nerede?

Bu plan **Giriş → Gelişme → Sonuç** düzenindedir. Önce **yapım sırası** (hangi adım, hangi sırayla), sonra **birleştirme** (iOS, API ve veritabanının nasıl bağlandığı) anlatılır. Teknolojilerin kendisi, algoritmalar ve iş mantığı **Yukum_Teknik_Rapor.pdf** dosyasındadır.

| Kısım | Bölüm | Konu | İçinde ne var? |
|---|---|---|---|
| **GİRİŞ** | 1 | Amaç, kesin kararlar, çalışma düzeni | Kim ne yapar, “karıştırmama” kuralı, kapsam sınırı |
| **GELİŞME I · YAPIM SIRASI** | 2 | Büyük resim | Aşama A–L özeti; ilk çalışır hâl = A–F |
| **GELİŞME I · YAPIM SIRASI** | 3 | Adım adım yapım planı | Her adım: ne yapılır (backend / veritabanı / iOS), “bitti sayılır” koşulu |
| **GELİŞME II · BİRLEŞTİRME** | 4 | Bağlama kuralları | JSON sözleşmesi, hata kodları, yerel bağlantı (ATS), dikey dilim döngüsü |
| **GELİŞME II · BİRLEŞTİRME** | 5 | Sözleşme taslakları | Ana DTO alan listeleri |
| **SONUÇ** | 6 | Notlar → adımlar eşleşmesi | Hiçbir şeyin atlanmadığının kontrolü, açık sorular, sıradaki adım |

> **Sıra ve numaralar için bağlayıcı doküman bu plandır.** Backend, veritabanı ve iOS teknolojilerinin ayrıntısı: Teknik Rapor Bölüm 5–12. Algoritmalar: Teknik Rapor Bölüm 13.

GİRİŞ

# 1. Amaç, Kesin Kararlar ve Çalışma Düzeni

## 1.1 Kesinleşen karar

> **K-005 KESİNLEŞTİ:** İlanı **Yük Veren** açar. **Taşımacı** ilana **fiyat teklifi** verir. Taşımacı ilan açamaz, yük veren teklif veremez. (Teknik Rapor'daki Q1 sorusu kapandı.)

> **Proje amacı:** İş mülakatında "iOS geliştiriciyim ama backend'i ve veritabanını da güncel teknolojilerle kurdum" diyebilmek için vitrin proje. Kapsam **yalnızca notlardaki** maddelerdir; teknoloji seçimleri araştırmayla doğrulandı (Teknik Rapor, Bölüm 16).

## 1.2 Çalışma düzeni

| Alan | Kim yapar | Nasıl |
|---|---|---|
| Backend (.NET) + Veritabanı (MSSQL) | Birlikte (Claude + kullanıcı) | Her adımda önce mantık anlatılır, kullanıcı onaylar, sonra küçük parça yazılır, Swagger'da denenir. Kullanıcı kodu okuyup anlar (öğrenme hedefi). |
| iOS (Swift / SwiftUI) | **Kullanıcı** (Swift'e hâkim) | Claude sözleşmeyi (endpoint, JSON, hata kodu) verir, tasarım/mimari önerir, istenirse kodu gözden geçirir. Kullanıcı yazar. |
| Bağlama | Birlikte | Her dikey dilimde "API hazır + Swagger'da kanıtlı" olmadan iOS adımına geçilmez. |

## 1.3 Karıştırmama kuralı (algoritma disiplini)

- **Önce ilk çalışır hâl (yürüyen iskelet).** Hedef birinci derecede, notta verilen teknolojilerle (iOS SwiftUI ↔ .NET Web API ↔ MSSQL) uçtan uca çalışan **en sade** sürümü çıkarmaktır: Aşama A–F. Tasarım cilası, ek özellikler ve mühendislik süslemeleri (Docker, CI, OpenTelemetry, bildirim, konum vb.) ilk çalışır hâlden **sonra**, istenirse tek tek eklenir. Bugün kesin olmayan kararlar (Q'lar) yol ilerledikçe, ilgili adımda birlikte verilir; başta hepsini çözmek gerekmez.
- **Tek seferde tek dilim.** Bir dilim bitmeden (kabul kriteri sağlanmadan) sonrakine geçilmez.
- **Sıra her zaman aynı:** Veritabanı → Repository → Servis (iş kuralı) → Controller → Swagger testi → otomatik test → iOS Endpoint/Model → ViewModel → View → cihazda deneme → Git commit.
- **Önce çalıştır, sonra süsle.** Güzelleştirme (Aşama L) işlevler bittikten sonra; ancak renk/font kararı baştan alınır.
- **Küçükten büyüğe:** Önce "Merhaba" (health) → referans veri (il/ilçe) → kimlik → ilan → teklif → iş/puan → bildirim → konum → yayın.
- **Sözleşme tek kaynaktır:** Swagger/OpenAPI. iOS modelleri ona göre; değişiklik önce sözleşmede yapılır.

> **Notta olmayan şeyler çıkarıldı:** Önceki sürümlerde benim eklediğim "Dönüş Rotası / Müsaitlik", rota eşleştirme bildirimi, ilan süresi bildirimi, mesajlaşma, belge doğrulama, ödeme, harita görünümü ve admin web paneli bu sürümde **yoktur**. Kapsam yalnızca notlarındaki maddelerdir. Tek istisna: App Store'a yayın yapılırsa Apple'ın zorunlu kıldığı şikâyet/engelleme/hesap silme (Aşama K1) — bu yalnız yayın kararı verilirse yapılır.

GELİŞME I · YAPIM SIRASI (büyük resim)

# 2. Büyük Resim: Aşamalar (Küçükten Büyüğe)

**Kilometre taşı 1 – "İlk çalışır hâl" = Aşama A → F.** Kayıt/giriş (2 tip) + otomatik oturum, il/ilçe, ilan verme/listeleme/detay, teklif verme/kabul. Basit ekranlar, sistem bileşenleri. **Sonrası (G–L) ilk çalışır hâl onaylandıktan sonra**, sırayla ve istenirse seçerek eklenir.

| Aşama | Konu | Backend/DB | iOS | Sonuç |
|---|---|---|---|---|
| A | Hazırlık | Kurulum, repo | Xcode | Boş ama düzgün iskelet |
| B | Backend temeli | Solution, DB, Repository | — | Swagger açılıyor, DB bağlı |
| C | İlk bağlantı kanıtı | İl/İlçe servisi | APIClient + il/ilçe ekranı | **iOS ↔ API ilk kez konuşuyor** |
| D | Kimlik | Register×2, login, refresh | Keychain, login/register, auto-login | Rol bazlı giriş + oturum hatırlama |
| E | İlan | Loads CRUD + liste/filtre | İlan ver, liste, detay | Yük veren ilan açar, taşımacı görür |
| F | Teklif | Offers, kabul/ret | Teklif ver / yönet | MVP |
| G | İş, puan, profil | Shipments, Reviews | İş durumu, yorum, profiller | Anlaşma → teslim → puan |
| H | Favori & sayaç | Favorites, views | Favori, sayaçlar | İlan sahibi ilgiyi görür |
| I | Bildirim | APNs, outbox, e-posta | Push izni, deep link | Anlık haberleşme |
| J | Konum & emsal fiyat | Spatial sorgu, emsal fiyat | CoreLocation, "bana yakın" | Konuma göre ilan bulma |
| K | Sertleştirme & yayın | Güvenlik, moderasyon, hesap silme | Gizlilik, TestFlight | App Store'a hazır |
| L | Süsleme & v2 | AI moderasyon (opsiyonel) | Animasyon, tasarım cilası | Cila |

GELİŞME I · YAPIM SIRASI (adım adım)

# 3. Adım Adım Yapım Planı

Numaralar yapım sırasıdır. DB veritabanı, API backend, iOS mobil. Her adımın sonunda **Bitti sayılır** koşulu sağlanmadan sonrakine geçilmez.

## AŞAMA A – Hazırlık

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| A1 | Kurulum: .NET 10 SDK, VS Code + C# Dev Kit, Docker + SQL Server 2022 konteyneri, Git, Postman (veya Scalar arayüzü), **Xcode 26** (iOS 26 SDK). | `dotnet --version` 10.x; SQL Server'a araçla bağlanılıyor. |
| A2 | Depo düzeni: `/backend`, `/ios`, `/docs` (iki rapor + karar kayıtları). `.gitignore`, README, ilk commit. | Git'te boş iskelet. |
| A3 | Ürün adı/bundle ID/namespace kararı ( `Yukum` ). Apple Developer hesabı planı (Aşama I'den önce açılmalı). | K-001 kararı yazıldı. |
| A4 | Tasarım **kararları** (yalnız seçim): ana renk, vurgu rengi, font (SF Pro), köşe yuvarlaklığı, ikon seti. Uygulama sonra (Aşama L). | Tasarım sayfası (1 sayfa) onaylı. |

## AŞAMA B – Backend Temeli (iOS'a henüz dokunulmaz)

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| B1 | API Solution ve 4 proje: Domain, Application, Infrastructure, Api + test projeleri; proje referansları (bağımlılık yönü doğru). `GET /health` ve Swagger. | Swagger açılıyor, `/health` 200. |
| B2 | API Ortak altyapı: Serilog, OpenTelemetry (temel; ertelenebilir), ProblemDetails global hata middleware'i, API versiyonlama ( `/api/v1` ), JSON ayarları (camelCase, enum string), **yerleşik OpenAPI + Scalar**, mapping (elle/Mapster), `user-secrets`. | Bilerek atılan hata ProblemDetails dönüyor; log dosyada. |
| B3 | DB `AppDbContext` , SQL Server bağlantısı, ilk migration, `dotnet ef database update` . Ortak taban sınıf (Id, CreatedAt, UpdatedAt, IsDeleted, RowVersion). | DB oluştu; health check DB'yi de doğruluyor. |
| B4 | API `IRepository<T>` + `IUnitOfWork` + DI kaydı; Application katmanında **CQRS-lite** handler yapısı (Commands/Queries, MediatR'sız). Birim test örneği. | Sahte bir entity ile ekle/oku testi geçiyor. |
| B5 | API **(İlk çalışır hâlden sonra yapılabilir) Dockerfile + docker-compose** (API + SQL Server) ve **GitHub Actions** (build + test). | `docker compose up` ile API+DB ayağa kalkıyor; CI yeşil. |

## AŞAMA C – İlk Bağlantı Kanıtı (il/ilçe)

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| C1 | DB `Cities` (81) ve `Districts` (~970) entity + migration + **seed** (resmî liste, doğrulanarak). Koordinatlar (merkez) eklenir (konum özelliği için). | Tablolarda 81 il, ilçe sayıları doğru. |
| C2 | API `GET /cities` , `GET /cities/{id}/districts` ; anonim; **cache**; sözleşme kartı. | Swagger'da liste; olmayan il için 404. |
| C3 | iOS **Xcode 26** projesi (Swift 6.2, approachable concurrency, **min iOS 18**), klasör yapısı (Teknik Rapor §10.3, MVVM + Router), `.xcconfig` base URL, Debug ATS/HTTPS ayarı. | Boş uygulama simulator'de açılıyor. |
| C4 | iOS **Generic APIClient**: `Endpoint` , `APIError` , tek genel `send<Response>` metodu, decoder ayarları (ISO8601). Henüz auth yok. | Birim test: sahte URLProtocol ile başarı + hata çözümü. |

| | | |
|---|---|---|
| C5 | iOS Test ekranı: il seçilince ilçeler yüklenir (cascade picker), yükleniyor/hata/boş durumları. | **Kilometre taşı:** simulator'de API'den gelen il-ilçe listesi görünüyor. |

> **Neden il/ilçe ile başlıyoruz?** Basit, anonim ve ileride (kayıt, ilan verme, filtre) gerekecek. Bağlama sorunları (ATS, base URL, JSON, tarih) burada en ucuz şekilde çözülür.

## AŞAMA D – Kimlik (Login / Register / Auto-login)

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| D1 | DB ASP.NET Identity (Guid) + `UserType`, `ShipperProfiles`, `CarrierProfiles`, `Vehicles`, `RefreshTokens`. Roller seed: `Admin`, `Shipper`, `Carrier`. Admin hesabı **yalnız seed/elle** (gizli şifre). | Migration uygulandı; roller tabloda. |
| D2 | API `POST /auth/register/shipper` ve `/register/carrier` (ayrı DTO'lar, rol endpoint'ten atanır), şifre kuralları, e-posta/telefon benzersizliği, KVKK onay alanı. | İki kayıt Swagger'da çalışıyor; aynı e-posta 409. |
| D3 | API `POST /auth/login`, JWT üretimi (10–15 dk), claim: sub, role, jti. Yanlış şifrede genel mesaj, 5 hatada geçici kilit. | Token dönüyor; `[Authorize]` uç 401/200 veriyor. |
| D4 | API Refresh token: üretim, **hash'li** saklama, `/auth/refresh` **rotasyon**, eski token tekrar kullanılırsa token ailesi iptal, `/auth/logout`. | Rotasyon + çalıntı senaryosu testle kanıtlı. |
| D5 | API Yetki: policy'ler (`ShipperOnly`, `CarrierOnly`, `AdminOnly`), rate limiting (login/register), `GET /me`. Entegrasyon testi: **yetki matrisi** (Teknik Rapor §6.4) her hücre için. | Carrier, Shipper uçlarında 403; anonim 401; testler geçiyor. |
| D6 | iOS `KeychainService`, `TokenStore`, `AuthInterceptor` (Bearer ekle), 401'de **actor** ile tek refresh + isteği tekrar dene. | Birim test: aynı anda 5 istek 401 alınca 1 refresh. |
| D7 | iOS Ekranlar: karşılama (Yük veriyorum / Taşımacıyım), Login, Register-Yük Veren, Register-Taşımacı; form doğrulama + sunucu `errors` eşleme. | İki tipte kayıt + giriş çalışıyor. |
| D8 | iOS **Auto-login** (Teknik Rapor §11.4): açılışta Keychain → refresh → rol bazlı kök (Shipper/Carrier TabView); ağ yoksa oturum silinmez; çıkış akışı. | **Kilometre taşı:** uygulamayı kapat-aç → giriş istemiyor; token iptalinde login'e düşüyor. |

## AŞAMA E – İlan (Yük Veren açar, herkes görür)

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| E1 | DB `Loads` (durum enum'u, il/ilçe FK, `geography` konum kolonları, sayaçlar, RowVersion) + indeksler (Teknik Rapor §9.4) + migration. | Migration temiz; indeksler SQL'de görünüyor. |
| E2 | API `POST /loads` (Shipper): doğrulama (ilçe↔il uyumu, tarih geleceğe ait, ağırlık>0), konum = ilçe merkezi, Status=Open, ExpiresAt. 201. | Shipper 201; Carrier 403; hatalı ilçe 422. |
| E3 | API `GET /loads/{id}` (detay; sahibine ek alanlar), `PUT /loads/{id}` (sahip + teklif kabul edilmemiş), `POST /loads/{id}/close`, `GET /loads/mine`. **Sahiplik testi:** başkasının ilanı 403. | IDOR testi geçiyor. |
| E4 | API `GET /loads` liste: sayfalama + filtre (kalkış/**varış il-ilçe**, fiyat aralığı, araç tipi, tarih) + sıralama (en yeni, fiyat artan/azalan). Dinamik predicate (Teknik Rapor §13.6). Konum/puan sıralaması **şimdilik yok**. | Tüm filtre kombinasyonları Swagger'da doğru sonuç veriyor. |
| E5 | iOS Shipper: **İlan Ver** (çok adımlı form, il→ilçe cascade), **İlanlarım**, ilan detayı (sahip görünümü). | Cihazdan ilan açılıyor, listede görünüyor. |

| | | |
|---|---|---|
| E6 | iOS Carrier: **İlan Listesi** (sonsuz kaydırma, pull-to-refresh), filtre sheet (nereden/**nereye**, fiyat), sıralama, detay (taşımacı görünümü). | **Kilometre taşı:** iki kullanıcıyla iki simulator/cihazda ilan açıp görme. |

## AŞAMA F – Teklif (Taşımacı fiyat verir) MVP SONU

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| F1 | DB `Offers` + filtreli benzersiz indeks (aynı taşımacı, aynı ilan, tek **aktif** teklif) + `Shipments` + migration. | Aynı teklif ikinci kez eklenemiyor. |
| F2 | API `POST /loads/{id}/offers` (Carrier): ilan açık mı, süresi geçmemiş mi, fiyat>0; `OfferCount++` aynı transaction'da. Aktif teklif varsa güncelleme. | Shipper 403; kapalı ilana 422. |
| F3 | API `GET /loads/{id}/offers` (yalnız ilan sahibi), `GET /offers/mine`, `/offers/{id}/withdraw`. | Başkası teklifleri göremiyor. |
| F4 | API `POST /offers/{id}/accept`: transaction + RowVersion; teklif Accepted, diğerleri Rejected, ilan Assigned, `Shipment` oluşur; çakışmada 409. `/reject`. | Eşzamanlı iki kabul testinde yalnız biri başarılı. |
| F5 | iOS Carrier: **Teklif ver** ekranı, **Tekliflerim**. Shipper: ilan detayında **gelen teklifler** listesi, kabul/ret. | Uçtan uca: ilan → teklif → kabul. |
| F6 | MVP bütünleme: hata/boş durumları, Türkçe metin gözden geçirme, TestFlight iç test. | **MVP kilometre taşı:** iki rolde tam akış gerçek cihazda. |

## AŞAMA G – İş Durumu, Puan/Yorum, Profiller

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| G1 | API Shipment durum akışı: `/pickup` (Carrier), `/deliver` (Carrier), `/confirm` (Shipper); geçersiz geçiş reddi; `GET /shipments/mine`, `/shipments/{id}`. | Durum makinesi testleri geçiyor. |
| G2 | DB API `Reviews`: yalnız teslim edilmiş işe, yalnız o işin yük verenine, iş başına tek yorum; puan 1–5; `CarrierProfiles.AverageRating/RatingCount` aynı transaction'da. Bayesian skor (Teknik Rapor §13.7). | İkinci yorum 409; yetkisiz 403. |
| G3 | API `GET /carriers/{id}` (ortalama puan, tamamlanan iş, kısa özet), `/carriers/{id}/reviews`, `/users/{id}/public-profile`, `GET/PUT /me` (iki rolün profilinde geçmiş işler). | Profil verileri doğru. |
| G4 | API Puana göre sıralama: `sort=shipperRating` / teklif listesinde taşımacı skoruna göre öne çıkarma. | Sıralama testi. |
| G5 | iOS İş detayı + durum butonları, Puan & Yorum yaz, Taşımacı profili (puan + yorumlar), Profilim (anlaşılan işler her iki rolde). | **Kilometre taşı:** bir iş baştan sona tamamlanıp puanlanıyor. |

## AŞAMA H – Favoriler ve Sayaçlar

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| H1 | DB API `FavoriteLoads` (idempotent ekle/çıkar, `FavoriteCount`), `GET /loads/favorites`. | Aynı ilan iki kez eklenince hata yok, sayaç doğru. |
| H2 | DB API `LoadViews`: tekil görüntülenme (kullanıcı+gün), sahibinin kendi görüntülemesi sayılmaz, `ViewCount` atomik artış. Sahibe detayda: görüntülenme/favori/teklif sayıları (öncelik: **görme** > teklif sayısı). | Aynı kişi aynı gün tekrar sayılmıyor. |

| | | |
|---|---|---|
| H3 | iOS Favori butonu, Favoriler sekmesi, sahibe sayaç kartları. | Sayılar iki cihazda tutarlı. |

## AŞAMA I – Bildirimler (Push + E-posta)

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| I1 | DB `DeviceTokens`, `Notifications`, `OutboxMessages` + `POST /devices`, `DELETE /devices/{id}`, `GET /notifications`, okundu işaretleme. | Uçlar çalışıyor. |
| I2 | API **Outbox deseni**: olay (ilan yayınlandı, teklif geldi, ilan favorilendi, teklif kabul/ret, teslim, yorum) → aynı transaction'da outbox → `BackgroundService` worker (tekrar dene, backoff). | Worker sahte gönderici ile testte çalışıyor. |
| I3 | API `INotificationSender` → APNs implementasyonu (.p8 token auth, HTTP/2); 410 token temizliği; favori bildirimi için spam sınırı/toplama. | Gerçek cihaza push düşüyor. |
| I4 | API E-posta: `IEmailSender` (SMTP/SendGrid), "ilan yayınlandı" şablonu; hassas bilgi içermez. | Mail teslim ediliyor. |
| I5 | iOS Push izni (değer gösterildikten sonra), APNs token kaydı, bildirime tıklayınca ilgili ekrana **deep link** (Router), Bildirimler ekranı, bildirim tercihleri. | **Kilometre taşı:** teklif verilince ilan sahibinin telefonuna bildirim geliyor, dokununca ilan açılıyor. |

## AŞAMA J – Konum ve Emsal Fiyat öncelik düşük

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| J1 | DB API Spatial index + `lat/lon/radiusKm` filtresi ve `sort=nearest` (STDistance). Kullanıcı konumu **saklanmaz**. | 5 km / 25 km sonuçları doğru. |
| J2 | iOS CoreLocation izni (ihtiyaç anında), "Bana yakın" seçeneği, yarıçap seçimi, "konumuma göre sırala". | Cihazda yakın ilanlar listeleniyor. |
| J3 | API **Emsal fiyat**: aynı (kalkış il, varış il, araç tipi) için son N gün tamamlanan işlerin medyanı; yeterli veri yoksa gösterilmez. | İlan detayında "bu rotada ortalama ₺X". |
| J4 | iOS Emsal fiyat göstergesi (ilan detayı). | Detayda "bu rotada ortalama ₺X" görünüyor. |

## AŞAMA K – Sertleştirme ve Yayın

| # | Yapılacak | Bitti sayılır |
|---|---|---|
| K1 | API **(Yalnız App Store'a yayın yapılırsa — Apple UGC/hesap silme kuralı)** `Reports`, kullanıcı engelleme, `DELETE /account` (hesap silme), admin uçları (`/admin/*`, yalnız Admin), AuditLog. Basit yasaklı kelime filtresi. | App Store UGC gereklilikleri karşılanıyor. |
| K2 | API Güvenlik gözden geçirme (Teknik Rapor §14.1 listesi), yük testi (liste/filtre), indeks/sorgu planı kontrolü, yedekleme planı, canlı ortam + gizli yönetimi, HTTPS. | Kontrol listesi tamam. |
| K3 | iOS Şikâyet/engelleme/hesap silme ekranları, Gizlilik/KVKK metinleri, Privacy Manifest, izin metinleri, Instruments ile liste performansı, erişilebilirlik, karanlık mod kontrolü. | Kontrol listesi tamam. |
| K4 | TestFlight dış test → App Store incelemesi (demo hesapları: biri Shipper, biri Carrier). | Yayın. |

## AŞAMA L – Süsleme ve İleri Sürüm

| # | Yapılacak |
|---|---|

| | |
|---|---|
| **L1** | Tasarım sistemi uygulaması (A4 kararları): renkler, tipografi, bileşen kütüphanesi, boş/hata ekranları, animasyon, mikro etkileşim. |
| **L2** | Notta opsiyonel olanlar (öncelik değil): yorum/ilan metni için **yapay zekâ ile yasaklı içerik kontrolü** (şüpheli → inceleme kuyruğu); Android/Web istemcilerinin aynı API'ye bağlanması. Başka özellik yok. |

# 4. Bağlama Kuralları (iOS ↔ API ↔ Veritabanı)

## 4.1 Zincir

SwiftUI View → ViewModel → Service → APIClient (generic) ⇄ HTTPS+JSON ⇄ Controller → Service (iş kuralı) → Repository / UoW → EF Core → MSSQL

## 4.2 JSON sözleşme kuralları (iki tarafın uyacağı)

| Konu | Kural |
|---|---|
| Alan adları | `camelCase` (ASP.NET varsayılanı). Swift `Codable` ile doğrudan uyumlu. |
| ID | GUID **string** (Swift: `UUID`). İl/ilçe ID'si `int`. |
| Tarih/saat | ISO 8601, **UTC** ( `2026-10-20T10:15:00Z` ). Swift tarafında decoder `.iso8601` . |
| Para | `decimal` → JSON sayı; Swift'te `Decimal` . Para birimi ayrı alan ( `"TRY"` ). |
| Enum | API'de **string** olarak ( `"Open"` , `"Carrier"` ) — sayı değil. Yeni değer gelirse iOS çökmesin diye Swift enum'da `unknown` durumu. |
| Null | Opsiyonel alan = `null` veya hiç gönderilmez; Swift'te `Optional` . |
| Sayfalama | `?page=1&pageSize=20` → `{ items:[…], page, pageSize, totalCount }` |
| Başarı kodları | 200 OK, 201 Created (+ `Location` ), 204 No Content. |
| Hata kodları | 400 doğrulama, 401 oturum yok/geçersiz, 403 yetkisiz rol/sahip değil, 404 yok, 409 çakışma (eşzamanlılık/çift teklif), 422 iş kuralı, 429 çok istek, 500 sunucu. |
| Hata gövdesi | ProblemDetails: `{ type, title, status, detail, errors:{alan:[mesaj]}, traceId }` → Swift `APIError` bunu çözer, `errors` alanı forma gösterilir. |
| Kimlik | `Authorization: Bearer <accessToken>` (yalnız anonim olmayan uçlar). 401 → tek sefer refresh → isteği tekrarla. |
| Sürüm | Yol: `/api/v1/…` . Kırıcı değişiklik = v2. |
| Dil | Hata mesajları Türkçe; `Accept-Language` ile ileride EN. |

## 4.3 Her uç için "sözleşme kartı" (şablon)

```
Uç:         POST /api/v1/loads
Yetki:      Shipper
İstek:      { title, description, cargoType, weightKg, originCityId, originDistrictId,
              destCityId, destDistrictId, pickupDate, budgetPrice?, vehicleType }
Başarı:     201 + LoadDetailDto
Hatalar:    400 (doğrulama), 401, 403 (Shipper değil), 422 (ilçe ile il uyumsuz)
iOS:        Endpoint.createLoad • CreateLoadRequest • LoadDetailDTO • CreateLoadViewModel
Kabul:      Swagger'da 201; Carrier token ile 403; geçersiz ilçe ile 422
```

## 4.4 Yerel geliştirme bağlantısı (sık takılınan yer)

| Durum | Ne yapılır |
|---|---|

| | |
|---|---|
| Simulator → Mac'teki API | `http://localhost:5xxx` . iOS ATS düz HTTP'yi engeller → **yalnızca Debug** için `localhost` ATS istisnası. Alternatif: `dotnet dev-certs https --trust` ile HTTPS kullanmak (tercih edilen, canlıya benzer). |
| Gerçek iPhone → Mac'teki API | API `0.0.0.0` 'da dinler; telefon ve Mac aynı Wi-Fi'da; `https://<mac-ip>:port` . Geliştirme sertifikası telefonda güvenilir yapılmalı veya Debug'da istisna. |
| Base URL | Debug / Staging / Release için ayrı `.xcconfig` ; kodda sabit URL yok. |
| SQL Server (Mac) | Docker'da SQL Server 2022 imajı (Apple Silicon'da emülasyon); alternatif uzak/Azure SQL. Bağlantı dizesi `user-secrets` 'te. |
| CORS | iOS için gerekmez; Web istemci eklenirse yalnız bilinen origin'ler. |
| Push (ileride) | APNs yalnız gerçek cihazda; Developer hesabı gerekir (Aşama I öncesi açılmalı). |

## 4.5 Tek bir dikey dilimin döngüsü (her adımda tekrarlanır)

1. **Sözleşme kartı** yazılır (4.3) ve onaylanır.
2. **DB:** Entity + yapılandırma + migration (SQL'e bakılır).
3. **Repository** metotları.
4. **Servis:** iş kuralları (yetki, durum geçişi, doğrulama) + birim test.
5. **Controller** + DTO + FluentValidation.
6. **Swagger/Postman'de elle test** (başarı + her hata kodu).
7. Git commit ("feat(api): …").
8. **iOS:** Endpoint + DTO → Service → ViewModel → View (sade) → simulator/cihaz testi.
9. Kabul kriteri onayı → Git commit ("feat(ios): …") → sıradaki dilim.

GELİŞME II · BİRLEŞTİRME (sözleşme / DTO)

# 5. Sözleşme Taslakları (ana DTO'lar)

Aşağıdakiler iki tarafın ortak anlaşacağı alan listeleridir; kesin hâli ilgili adımın sözleşme kartında (4.3) onaylanır.

| DTO | Alanlar |
|---|---|
| RegisterShipperRequest | fullName, companyName?, email, phone, password, taxNumber?, cityId, kvkkAccepted |
| RegisterCarrierRequest | fullName, email, phone, password, vehicleType, capacityKg?, plateNumber?, homeCityId, kvkkAccepted |
| LoginRequest | emailOrPhone, password, deviceId, deviceName |
| AuthResponse | accessToken, accessTokenExpiresAt, refreshToken, user{ id, displayName, role, avatarUrl? } |
| CreateLoadRequest | title, description, cargoType, weightKg, volume?, vehicleTypeRequired, originCityId, originDistrictId, destCityId, destDistrictId, pickupDate, deliveryDeadline?, budgetPrice?, currency |
| LoadListItemDto | id, title, originCity/district, destCity/district, pickupDate, budgetPrice?, currency, vehicleType, weightKg, status, shipper{ id, displayName, rating? }, offerCount*, isFavorite, distanceKm? |
| LoadDetailDto | LoadListItem alanları + description, volume?, deliveryDeadline?, referencePrice? (emsal), isOwner; **yalnız sahibe:** viewCount, favoriteCount, offerCount, offers[] |
| CreateOfferRequest | price, currency, note?, estimatedPickupDate? |
| OfferDto | id, loadId, price, currency, note?, status, createdAt, carrier{ id, displayName, rating, ratingCount, completedJobs } |
| ShipmentDto | id, load{…özet}, agreedPrice, status, agreedAt, pickedUpAt?, deliveredAt?, counterpart{ id, displayName, phone? (anlaşma sonrası) }, canReview |
| CreateReviewRequest | rating (1–5), comment? |
| CarrierProfileDto | id, displayName, avatarUrl?, about?, averageRating, ratingCount, completedJobCount, vehicles[], recentReviews[] |
| PagedResult<T> | items[], page, pageSize, totalCount |
| ProblemDetails | type, title, status, detail, errors{}, traceId |

*Teklif sayısı herkese gösterilir mi, yalnız sahibe mi — açık karar (Q2).

SONUÇ · NOTLAR → ADIMLAR EŞLEŞMESİ

# 6. Tekrar Kontrol: Notlar → Adımlar Eşleşmesi

Notunuzdaki **her cümle** aşağıda bir adıma bağlıdır. “Yapılmadan bırakılan” madde yoktur; ertelenenler notta zaten “öncelik değil / opsiyonel” denilenlerdir.

| Notlardaki istek | Adım |
|---|---|
| Proje amacı: yük ilanı pazar yeri; taşımacı yük arar; teklifli açık pazar | E, F |
| Web API, RESTful, Android+Web+iOS aynı API | B1–B2, 4.2 (client-agnostic sözleşme) |
| .NET 10 / .NET Core, MSSQL, VS Code | A1, B1, B3, Teknik Rapor §16 |
| Login’de iOS → API; kontroller API’de; başarılıysa bilgiler JSON döner | D3, D8, 4.2 |
| Tüm CRUD API’de, JSON döner, SwiftUI entegre | Tüm B–K, 4.5 |
| İki kullanıcı tipi, iki ayrı register, birbirinin servis/ekranına girmez, API authentication, admin’e giremez | D1, D2, D5, D7 |
| EF Core + Repository Pattern | B3, B4 |
| iOS: SwiftUI, güncel pattern’ler, iş ilanı incelemesi | Teknik Rapor §16, C3 |
| Genel GET/POST/PUT servis metodu (request/response modeli ile dinamik) | C4 |
| Hazır/güzel tasarım; önce çalışsın sonra süsle | A4, L1 |
| Auto-login / beni hatırla, en sağlam yöntem | D4, D6, D8 |
| Login/Register | D |
| Yük ilanı verme sayfası (yalnız yük veren) | E2, E5 |
| İlan listeleme sayfası | E4, E6 |
| Konuma göre filtre; “bana yakın 5 km”; öncelik değil | J1, J2 |
| İlan kaydetme / favori | H1, H3 |
| İlan sonrası mail veya push | I4, I3 |
| Görüntülenme ve teklif sayısı (görme öncelikli) | H2 |
| İlana teklif verme sayfası; fiyatı taşımacı verir | F2, F5 |
| İlanı iş veren verebilir, iş arayan veremez | D5, E2 (403 testi) |
| Profilde daha önce anlaşılan işler; iki tarafın profilinde de | G1, G3, G5 |
| Filtre/sıralama fiyata, konuma, emsal fiyata, gideceği yere göre | E4, J1, J3 |
| Şehir/ilçe seçimi; hazır tablo + servis | C1, C2, C5, E5 |
| Profesyonel iş akışı; backend yapay zekâ ile, iOS mantığı bilinmeli | 1.2, 4.5 |
| Yük beğenilince / teklif verilince push; ilan detayında verileri görme | I3, I5, E3, H2 |
| Teslim edildi statüsünden sonra yük veren puan/yorum; puana göre öne çıkarma | G1, G2, G4 |
| Taşımacı profilinde ortalama puan ve geçmiş iş yorumları | G3, G5 |
| Opsiyonel: AI ile yasaklı kelime kontrolü | K1 (basit), L2 (AI) |
| “Yapay zekâya ver-yap yok”; rapor, yol haritası, adım adım; algoritma; tasarım seç; ekran ekran | Bu doküman, 1.3, 4.5, A4, tüm aşamalar |
| Veritabanı bağlantısı, Core projesi, iOS dosya/proje yapısı sektöre uygun | B1, B3, C3, Teknik Rapor §5, §10 |

**Hâlâ netleşmesi gereken kalan sorular (Teknik Rapor §20):** Q2 teklif görünürlüğü, Q3 bütçe fiyatı zorunlu mu, Q4 karşı teklif, Q5 telefon paylaşımı zamanı, Q8 anlaşma sonrası iptal, Q12 minimum iOS, Q13 yalnız MVVM mi VIPER denensin mi. Bunlar ilgili adıma gelmeden **konuşularak** karara bağlanacak (öneriler Teknik Rapor'da).

**Sıradaki somut adım:** A1 (kurulum kontrolü) ve A4 (tasarım kararları). Ardından B1 ile backend iskeletine başlıyoruz; iOS'a C3'te giriyoruz.